MyStreeT (Foundation) – Capstone Project for Certification

Contents

Purpose

Build a **basic sneaker shopping web application** that allows users to **browse sneakers, view details, add to cart, and place an order**. Include a **simple admin interface** to manage products. The focus is

on **foundational full-stack skills**—React/Angular, Spring Boot, REST, PostgreSQL, Git, and basic testing—aligned with the Foundation skill matrix.

---

1) About Us

At **MyStreeT**, we love sneakers and want to make it easy for users to explore and buy shoes they like. This **Fundamental** version prioritizes a smooth, simple experience with essential features only.

---

2) Problem Statement

Sneaker buyers need a simple way to **discover products, view essential details, and place orders**—without the complexity of payment gateways, notifications, or advanced account features.
Goal: Deliver a **minimal yet complete** shopping flow demonstrating full-stack fundamentals.

---

3) Roles

1. **Guest/User** – Can browse, view details, add to cart, checkout (login/registration required at checkout).
2. **Admin** – Can add/edit/delete products.

Note: No role-based permissions beyond a simple "admin" flag.

**** Simple "Admin Flag" Approach**

- **User Entity** includes a boolean field:

boolean isAdmin;

- When a user registers, isAdmin defaults to false.
- For the first admin, you can **seed** a user with isAdmin = true in the database.

Backend Concept

- Instead of implementing a full RBAC system (roles table, permissions, JWT claims with scopes), we just:
  - Check isAdmin in the controller/service for admin-only actions like adding or deleting products.
  - Example:
    - If isAdmin == true → allow product CRUD.
    - Else → return 403 Forbidden.

Frontend Concept

- After login, store isAdmin in the user context or local storage.
- Use it to:
  - Show/hide **Admin menu** (e.g., "Manage Products" link).
  - Prevent navigation to admin routes if isAdmin is false.

---

4) In-Scope Features

A) Authentication (Basic)

- Register, Login, Logout.
- Simple password hashing.

B) Product Catalog

- List products with name, price, thumbnail.
- Filter by brand and size.
- Product detail page with description, price, sizes, image(s).

C) Shopping Cart

- Add/Remove items: quantity update.
- Cart value calculation.
- **Persist cart in localStorage** (frontend) or via simple backend session—choose one.

D) Checkout

- Enter shipping address.
- **Mock payment** (radio: "Cash on Delivery" or "Mock UPI") → always succeeds.
- Create order with status = PLACED.

E) Admin – Product Management

- CRUD for products (name, description, price, sizes, stock qty, image URL).
- No reports, no inventory rules beyond decrement on order.

---

5) User Flows (Happy Path)

1. **Browse:** Guest opens site → sees product list → clicks a product.
2. **Details:** Views details → selects size → adds to cart.

3. **Cart:** Opens cart → adjusts quantity → proceeds to checkout.
4. **Auth:** If not logged in → prompted to register/login.
5. **Checkout:** Enters shipping info → selects payment = *Mock* → places order.
6. **Confirmation:** Sees Order Confirmation page (Order ID).
7. **Admin:** Admin logs in → adds/edits/deletes products.

---

7) Architecture

- **Frontend:** React (Hooks + Context API) *or* Angular (Services + Components). Routing enabled.
- **Backend:** Spring Boot (REST Controllers, Services, Spring Data JPA).
- **Database:** PostgreSQL (Dev may use H2 for quick start).
- **Auth:** Stateless JWT **or** simple session cookie (choose one; document trade-offs).
- **State:** Cart in localStorage .
- **Deployment:** Localhost

---

8) Tech Stack

Frontend

- React + Vite/CRA (Routing, Hooks, Context) **or** Angular (Routing, Services)
- HTML5, CSS3, JavaScript (ES6), TypeScript

Backend

- Java 17+, Spring Boot, Spring Web, Spring Data JPA, Validation
- JUnit5, Mockito

Database

- PostgreSQL, JPA/Hibernate
- H2 for local

Tools & Practices

- Git Basics (branching, PRs), Postman/Swagger, Agile ceremonies
- AI Pair Programming: GitHub Copilot/ChatGPT for boilerplate and explanations

---

9) Data Model

Entities & Relationships

- User (1) — (M) Order
- Order (1) — (M) OrderItem
- Product (1) — (M) OrderItem

Note: Use a simple sizesCsv like "7,8,9,10" to keep schema simple at Fundamentals.

---

10) API Outline

Auth

- POST /api/auth/register – {email, password} → 201
- POST /api/auth/login – {email, password} → 200 {token or session}
- POST /api/auth/logout – 204 (if session-based)

Products

- GET /api/products – list with optional ?brand=&size=
- GET /api/products/{id}
- POST /api/products *(admin)* – create
- PUT /api/products/{id} *(admin)*
- DELETE /api/products/{id} *(admin)*

Orders

- POST /api/orders – {items[], shippingAddress, paymentMode:"MOCK"} → creates order
- GET /api/orders/mine – list current user's orders
- GET /api/orders/{id} – view order details

Error Format (Consistent)

JSON

```
{
 "timestamp": "2025-09-12T12:00:00Z",
 "path": "/api/orders",
 "error": "VALIDATION_ERROR",
 "message": "Shipping address is required"
```

}

---

11) NFRs & SLAs

- **Performance:**
  - Local response time < **1s** for API calls under normal dev conditions.
  - Page load (first meaningful paint) < **5s** on localhost.
- **Availability:** Best effort (developer machine).
- **Security:**
  - Hash passwords (e.g., BCrypt).
  - Basic input validation & server-side checks.
- **Scalability/Concurrency:** Test with 5–10 concurrent requests locally. *Simulate 5–10 quick requests using Postman Runner or multiple browser tabs. No JMeter or load-testing tools required at this level.*
- **Error Handling:** User-friendly error toasts/messages on UI; structured error JSON on API.
- **Logging:** Console logs; simple request/response logs (no external logging infra).
- **Documentation:**
  - README.md with setup steps, seed data, and run commands.
  - API contracts documented with Swagger/OpenAPI.

---

12) Testing

- **Backend Unit Tests:**
  - Services & Repos with JUnit5 + Mockito
  - Target coverage: **≥60%** (method-level)
- **API Testing:** Manual via Postman or Swagger.
- **Frontend:** Optional unit tests for 1–2 key components (e.g., Cart reducer).
- **No automated regression/e2e** required.

---

13) Engineering PoV

- **Code Quality:**
  - Follows SOLID basics where practical (e.g., service layer, DTOs).
  - No critical lint errors (ESLint/TSLint).
- **Branching:**
  - main (stable), feature/* branches; PR reviews by peers/mentor.
- **Coverage:**
  - Aim 60–70% backend, frontend minimal.
- **Dependencies:**
  - Keep list short; beware of transitive vulnerabilities.
- **Docs:**
  - API endpoints documented; Postman collection included.

---

14) Delivery Plan **(3 Sprints, 1–2 weeks each)**

Sprint 1: Foundations & Catalog

- Project scaffolding (FE + BE), DB setup (H2→Postgres), Git workflow
- Product entity, repository, REST endpoints (GET /products, GET /products/{id})
- Frontend: Home + Product List + Product Detail
- Seed data loading

Sprint 2: Auth & Cart

- Register/Login; protect admin routes
- Cart (localStorage) + Add/Remove/Quantity
- Validation + basic error handling

Sprint 3: Checkout & Admin CRUD

- Checkout flow + Mock Payment → Create Order
- GET /orders/mine, Order Confirmation page
- Admin Product CRUD screen
- Polish: README, Postman collection, unit tests to ≥60%

---

15) Sample Acceptance Criteria

Product Listing

- Given I am a guest, when I open /, I see at least 8 seeded products with name, price, image.
- Filters (brand/size).

Auth

- Given I register with a new email, I can log in and remain logged in across refresh (token/session stored).

Cart

- Items persist in localStorage; quantity changes update totals instantly.
- Removing last item shows empty-state message.

Checkout

- Required fields validated; order created with status PLACED.
- Confirmation page shows Order ID and item summary.

Admin

- Only admin can access /admin/products.
- Admin can add a product and see it on the catalog immediately.

---

16) Example Seed Data (SQL or JSON)

SQL

-- Minimal seed (PostgreSQL)

INSERT INTO users(id,email,password_hash,is_admin,created_at)

VALUES (gen_random_uuid(),'admin@mystreet.com','$2a$10$hash_here',true,now());

INSERT INTO product(id,name,brand,description,price,image_url,sizes_csv,stock_qty,created_at)

VALUES

(gen_random_uuid(),'Air Max 90','Nike','Classic retro vibe',119.99,'https://picsum.photos/seed/airmax/400','7,8,9,10',50,now()),

(gen_random_uuid(),'Ultraboost','Adidas','Responsive cushioning',139.99,'https://picsum.photos/seed/ub/400','7,8,9,10,11',35,now());

17) Simple Wireframe

- **Home/List:** Grid of cards → [Image] Name – Price – “View” / “Add to cart”
- **Detail:** [Large Image] Title – Brand – Price – [Size select] – [Add to cart]
- **Cart:** Table (Item/Size/Qty/Price) – Total – [Checkout]
- **Checkout:** Shipping Form – Payment (Mock) – [Place Order]
- **Admin Products:** Table + “Add Product” form

---

18) Sample User Stories

- As a **guest**, I want to browse sneaker listings so I can discover products.
- As a **user**, I want to add items to my cart and change quantities so I can finalize my selection.
- As a **user**, I want to complete a simple checkout with a mock payment so I can place an order.
- As an **admin**, I want to add or edit products so I can maintain the catalog.

---

19) Definition of Ready (DoR) & Done (DoD)

DoR

- User story has clear acceptance criteria.
- API/DB changes identified (if any).
- Mock/UX reference exists (even a sketch).

DoD

- Code compiles; unit tests for backend services pass.
- Manual tests completed (Postman + UI).
- No critical ESLint/TS issues; meaningful README update.
- Peer review completed; merged to main.

---

20) Evaluation Rubric (Aligned to Foundation Skills)

Area	What We Check	Weight
Frontend (React/Angular)	Routing, hooks/services, component communication, state handling (Context/Service), basic TypeScript usage	25%
Backend (Spring Boot)	Clean REST controllers, services, JPA entities & relationships, validation, basic error handling	25%
Common Skills	Core Java correctness, SQL/JPA usage, SOLID basics, Git hygiene, HTML/CSS semantics	20%
Testing	JUnit/Mockito for services/repos; Postman collection; target ≥60% coverage	15%
Generative AI Usage	Appropriate use of Copilot/ChatGPT for boilerplate, explanations, refactoring suggestions (cite prompts in README)	10%
Documentation & Readme	Setup steps, seed data, API list, constraints & trade-offs	5%

---

22) Stretch Goals (Optional, If Time Permits)

- Pagination on product list.
- Basic search bar.
- Simple input masking/formatting for address.

---

23) Getting Started (Suggested Commands)

Backend

- Java 17+, Spring Boot.
- Profiles: dev (H2), prod (Postgres).

Frontend

- React: npm create vite@latest mystreet-ui -- --template react-ts
- Angular: ng new mystreet-ui --routing --style=scss

Run Order

1. Start DB (H2 or Postgres) → 2) Run Spring Boot → 3) Seed Data → 4) Run UI

==========